Applied Intelligence (2023) 53:29282–29299
https://doi.org/10.1007/s10489-023-05083-5

Temporal enhanced inductive graph knowledge tracing

Donghee Han[1] · Daehee Kim[1] · Minsu Kim[2] · Keejun Han[3] · Mun Yong Yi[2]

Accepted: 2 October 2023 / Published online: 26 October 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract
The COVID-19 pandemic has led to a surge in online education, yielding increased attention on knowledge tracing (KT). KT involves predicting a student's mastery of knowledge based on the student's learning activity and plays a crucial role for recommending appropriate educational content at the individual level. While several deep learning-based KT models have been proposed, most consider only the interaction sequence of the target student, ignoring the interactions of other students. Furthermore, in practice, all model graphs should be incremented as more student interaction data is obtained, implying that they should be rebuilt to remain up-to-date, which is time-consuming. Moreover, existing graph-based KT models typically rely on predefined knowledge graphs for graph-based learning. Constructing a knowledge graph is labor-intensive, requiring a higher understanding of the domain knowledge. To address these problems, we propose a new inductive graph-based KT (IGKT) framework, which fully utilizes a subgraph extraction approach from a student-exercise bipartite graph, promising scalability and efficiency for practical applicability. The proposed framework consists of three key components: subsequence extraction, subgraph extraction and exericse information addition, and graph-based correctness prediction. For the third component, we test three variants to determine the effectiveness of integrating the timestamp and graph information, thereby creating three models for comparison: IGKT-Basic, IGKT with timestamps (IGKT-TS), and IGKT with graph attention layers that incorporate timestamps as edge features (IGKT-GAT). Extensive experiments conducted on multiple real-world datasets show that the models based on the proposed framework are superior over state-of-the-art models, and IGKT-GAT is the most effective implementation of the proposed framework. We further validate the effectiveness of the IGKT models through an ablation study and multiple in-depth analyses.

Keywords Knowledge tracing · Graph-based knowledge tracing · Inductive graph · IGKT

1 Introduction

Because of the COVID-19 pandemic, the learning environment has rapidly shifted from offline to online education. The online learning environment has become not only a supplementary tool but also the primary teaching method for efficient learning. As online learning is performed on various learning management systems (LMSs), the learning and activity logs of students can be tracked, enabling personalization for individual learners. However, tracking the educational gap among learners has become more difficult because of relatively fewer interactions between instructors and learners during online classes.

To solve these issues, there have been several attempts to develop methods for measuring understanding based on the student activity logs. One such method is knowledge tracing (KT), which measures the degree of understanding of specific knowledge concepts (KCs) based on a student's activity record [1]. In particular, KT predicts the probability of whether a student can answer a given problem correctly based on their learning activity sequence. Each learning activity contains specific knowledge concepts, often described as a set of domain representative terms, which play a vital role in the prediction [2].

As KT is a prediction task, the technique can be employed alongside learning exercise recommendation [1]. In general, instructors are required to navigate through groups of students and manually select the appropriate exercises for each student's level of knowledge to ensure personalized learn-

✉ Keejun Han
keejun.han@hansung.ac.kr

✉ Mun Yong Yi
munyi@kaist.ac.kr

Extended author information available on the last page of the article

ing. Selecting exercises that are too easy or too challenging hinders effective learning. To avoid this problem, a model that adjusts the difficulty of the exercises corresponding to the student's knowledge status was proposed [3]. Moreover, adaptive learning, which actively personalizes the learning contents or the manner in which they are presented, has also been proposed for KT to constantly measure the students' knowledge status [4]. In sum, KT is crucial for suggesting suitable learning contents and personalized learning paths, and it should consider various factors such as knowledge acquisition, forgetting, and the relationships between KCs in the learning process.

To develop an efficient end-to-end approach that covers all of the aforementioned factors, deep learning-based models, such as Deep Knowledge Tracing (DKT) have been proposed by adapting successful deep-learning approaches from various domains. For instance, sequence models such as recurrent neural networks (RNNs) and self-attention-based models such as SAKT [5] and SAINT [6] have been devised, which utilize transformer-based models for sequence-based prediction tasks. Another study confirmed that DKT does not handle long sequences well through finite state automation; the study proposed a model that utilizes a combination of self-attention and long short-term memory (LSTM) to capture the global dependency relationships of the input [7]. Recently, as research on graph neural networks (GNNs) has gained attention, a graph-based KT (GKT) model that learns the relationships between KCs has also been introduced [8].

However, there are still several limitations in existing KT models. First, existing models track knowledge for an individual student but do not consider the difficulty or relevance of exercises that may be discovered by other students' interactions. Second, the models struggle to extract high-order information, which can be learned by utilizing the interactions of neighboring students. High-order information refers to information that can be captured from multi-hop neighbors beyond the immediate node. Recommendation system studies that utilize user-item relationships have shown the importance of discovering high-order information [9–13]. Existing studies have used GNNs to discover various neighboring nodes and capture higher-order connectivity through multi-hop in user-item graphs. Within the research area of KT, however, there has been a lack of attention on utilizing such high-order connectivity information. Graph-based approaches have attempted to address this issue by representing relationships between KCs as a graph and exploiting it in a sequence-based model [8, 14, 15]. However, these methods remain inefficient in terms of time complexity when the density of the KC graph increases. In practice, the graph evolves as the student interaction data continues to increase. The entire graph should then be rebuilt to remain up-to-date, which is time-consuming. Furthermore, these GKT models typically rely on predefined exercise-KC graphs for graph-based learning. Constructing an exercise-KC graph is labor-intensive, requiring a higher understanding of the domain knowledge.

To overcome the limitations of existing KT models, we propose a new KT framework based on a student-exercise bipartite graph called Inductive Graph-based Knowledge Tracing (IGKT). Figure 1 illustrates a core concept of our approach, which considers other students' interactions. Previous graph-based approaches have modeled the connections between KCs as a graph; however, our method adds connection information between KCs to a bipartite graph composed of student-exercise interactions. In addition to the basic IGKT model that implements the proposed framework, we develop two extended models that incorporate temporal and sequential information, moving beyond existing graph-based rating prediction models such as IGMC [16] because the order of interactions is a significant factor in the KT tasks [2].

The proposed framework has three advantages. First, it can learn about the target student's interactions as well as those of other students regarding the target learning activity. Second, it is robust for large web-scale graphs using a subgraph-based method, as it does not require additional learning when applied to an evolving graph. Finally, it combines temporal information and graph structures to enable the integration of multiple KCs with student-exercise interactions.

In summary, the contributions of this study are as follows:

- We propose an inductive graph-based KT framework that utilizes time information and KCs in a student-exercise graph.
- In addition to the basic model that implements the proposed framework, we develop two extended models that additionally utilize temporal information and compare their effectiveness.
- We analyze the impact of the correlation between KCs on the performance of a graph-based KT model.
- We conduct experiments on multiple datasets to demonstrate the superior performance of the proposed framework.

The remainder of this paper is organized as follows. In the related work section, we review prior research on KT and GNNs. In the method section, we present the details of the proposed framework and models. The experiment section outlines the research questions and presents the experimental results. The analysis section discusses the characteristics of our model based on the experimental results, and the conclusion section summarizes the findings of this study.[1]

[1] A preliminary version of this paper was presented at the IEEE International Conference on Big Data and Smart Computing (BigComp 2023)

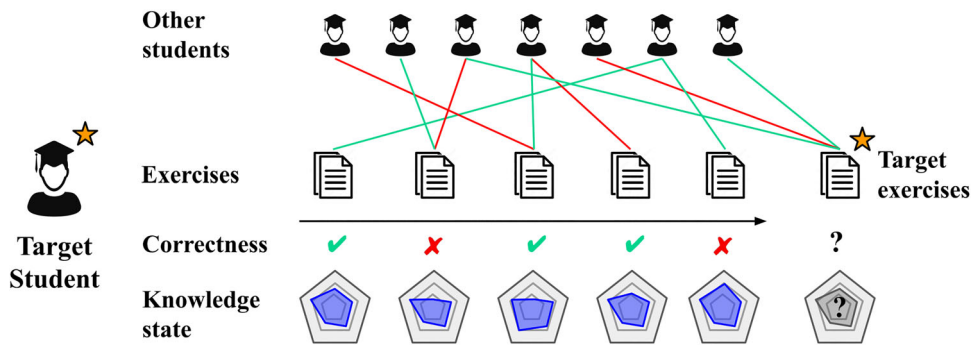

Fig. 1 Knowledge tracing considering other students' interactions

2 Related work

2.1 Knowledge tracing

Several studies have focused on KT models. Traditional machine learning-based approaches, such as Bayesian KT (BKT) [17], use a hidden Markov model to update student skills in a binary knowledge state.

Deep learning-based approaches, such as Deep KT (DKT) [18], use an RNN to update student knowledge states into a hidden state based on the student's interaction sequences. While DKT has demonstrated an improved performance, it is limited in expressing a student's learning state as a hidden state, without the distinction of specific concepts.

The Dynamic Key-Value Memory Networks (DKVMN) [19], an extension of DKT, uses a memory network to track the knowledge state of each student for each KC, thereby improving the model interpretability. However, this model also has limitations, including treating each interaction with the same weight.

To improve upon previous models, attention mechanisms have been introduced, such as in the SAKT [5] and SAINT [6] models. SAKT uses attention layers to learn the importance of each exercise in past interactions for predicting whether a student will correctly answer a target exercise. SAINT is a model that improves upon SAKT by using a transformer-based method with both an encoder and a decoder as well as the student's interaction sequence including their exercise and response data. SAINT+ [20] is a variant of SAINT that adds two features: the time the student spent answering each question and the time interval between two continuous learning interactions. In addition, various KT models have been proposed to reflect changes in students' knowledge status over time [21, 22].

Some research has also explored the use of graph representation learning techniques, such as GNNs, which can capture complex relationships such as the similarity and dependency between KCs. For example, GKT [8] learns the relationship between KCs using a graph structure, where KCs are represented as nodes and their dependencies as edges. However, our proposed model is different in that it can utilize student-exercise interactions and incorporate various types of KCs into the model.

Recently, several studies have proposed methods for creating richer features using exercise text [14, 23]. Studies have used BERT [24] to convert exercise text into vectors to capture the semantic relationships between exercises. Neural architecture search (NAS) has also been proposed to explore suitable structures for integrating multi-modals and multi-modal fusion architectures that can automatically filter noise in features [25].

2.2 Graph neural network

The use of graph data structures for expressing user-item interactions has been popular in recommender systems in various domains. Recent research has focused on using GNNs to generate user and item representations from interaction data.

GCMC [26] is a transductive model for matrix completion tasks. It uses a graph convolutional autoencoder to obtain user and item representations from the rating matrix. GCMC uses message passing on the bipartite interaction graph to generate latent user and item representations, which are then used in a bilinear decoder to reconstruct the rating edges.

GraphSAGE [27] is an early inductive graph framework that efficiently creates node representations from random-walk subgraphs using node feature information such as text and image properties. Instead of training individual embeddings for each node, it learns the topological structure of each node's neighborhood as well as the distribution of node features in the neighborhood by including node features in the learning process.

PinSAGE [28] is a multi-modal GNN model that applies GraphSAGE to the problems of the Pinterest. Image and text data are used to create node features for learning, and additional learning techniques such as importance pooling and curriculum training for graph learning are used to achieve high performance. PinSage, although inductive, heavily relies on the rich visual and linguistic material connected with the pins, which is not always available in other recommendation tasks.

Subgraph-based inductive methods such as IGMC [16] can be used to generalize tasks and can be applied to unseen nodes. IGMC extracts a local enclosing subgraph for each target user-item pair, consisting of the k-hop neighbor nodes and edges of the target user-item pair. The GNN learns complex patterns of the graph and predicts the rating. Unlike early inductive graph frameworks that rely on node features, IGMC does not use side information and utilizes only the graph structure and ratings. In IGMC, the relational graph convolutional network (R-GCN) [29] layers are used to generate the subgraph representations. Unlike IGMC [16], our study incorporates time and sequence information, given the importance of the order of interactions in the KT tasks [2]. Additionally, the graph attention network (GAT) [30] is a GNN model that uses self-attention mechanisms to learn the importance of neighbors in a graph, allowing for effective representation learning in large-scale graphs. The edge graph attention network (EGAT) [31] is an extension of the GAT that incorporates edge features and attention mechanisms to better capture complex relationships in the graph.

3 Method

In this section, we introduce our proposed IGKT framework and its components. After introducing the overall structure of the framework, we describe each component in detail.

3.1 Problem formulation

This study aims to predict whether a student will correctly answer a target exercise based on their learning sequence. The student's interaction sequence is divided into non-overlapping subsequences of fixed length. Formally, the interaction sequence of a student is denoted as $I = \{i_0, i_1, i_2, \ldots, i_T\}$. Each interaction contains the exercise e_t and correctness c_t at time t. The exercise e_t contains the ID of the exercise and related KCs. Correctness $c_t \in \{0, 1\}$ equals 1 if the student's answer is correct and 0 if incorrect. The problem formulation has been defined similarly in most KT tasks [1]. In terms of probability estimation, the problem can be expressed as shown in (1) as,

$$P(c_t = 1 \mid i_0, i_1, \ldots, i_{t-1}, e_t). \tag{1}$$

3.2 Overview

Figure 2 presents the proposed IGKT framework, which consists of the following three steps.

Subsequence extraction: The first step of the IGKT is to extract a fixed-length subsequence of the student's previous interactions. This subsequence represents the student's learning history and contains information regarding which exercises the student has completed. The extracted subsequence is used to create the subgraph in the subsequent step.

Subgraph extraction and exercise information addition: The second step involves extracting a subgraph from the student's previous interactions and adding additional information to it in the form of edges. The subgraph consists of the target student, target exercise, and the 1-hop neighbors of each. To capture the relationships between the exercises in the subgraph, we add edges between exercises belonging to the same problem type or sharing skills. These edges represent the co-occurrence of KCs and can help the model in predicting the student's performance on the target exercise.

Graph-based correctness prediction: In the final step, we use the subgraph from the previous step to predict the student's performance on the target exercise. First, node embeddings are derived from the subgraph using GNN layers. These node embeddings capture the relationships between the student and the exercises in the subgraph, as well as the co-occurrence of KCs represented by the added

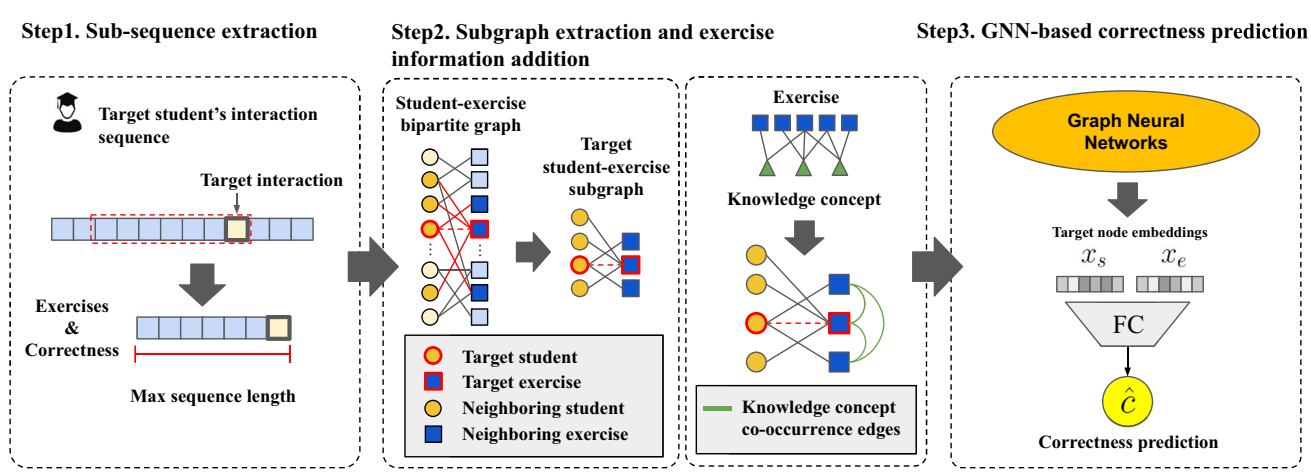

Fig. 2 Inductive graph-based knowledge tracing framework

edges. Using the node embeddings, the correctness of the student's answer is then predicted, allowing us to effectively utilize the graph structure and additional information in the subgraph to make predictions regarding student performance on the target exercise.

3.3 Sub-sequence extraction

In the first step, a fixed-length subsequence of the student's previous interactions is extracted. This subsequence represents a segment of the student's learning history and includes timestamps and information regarding the exercises they have completed.

When predicting the correctness c of interaction i_t at time t, the prediction proceeds through a subsequence consisting of the closest $n-1$ interactions among the previous sequences. Prior studies have also used this subsequence-based learning method [6, 20].

Unlike some self-attention-based KT models that rely solely on order information and timestamps, the IGKT framework is designed to utilize both the characteristics of the graph and time-dependent information, allowing IGKT to predict student performance more effectively by considering both the relationships between exercises and the order in which they are completed.

3.4 Subgraph extraction and exercise information addition

In the second step, a subgraph is extracted from the student-exercise interaction graph and additional information regarding the exercises is added to the subgraph. It consists of nodes representing students and exercises, with edges representing the interactions between them, which are assigned a label; a value of 1 indicates that the student correctly solved the associated exercise, and a value of 0 indicates otherwise.

A list of candidate nodes for the subgraph is created based on the subsequence from the second step, which includes the target student and n exercises from the sequence. $n-1$ neighboring student nodes of the target exercise node are randomly selected from the interaction graph and are added to the list. The subgraph is then extracted from the interaction graph, including the target student, the attempted exercise, their neighboring nodes, and interaction edges with their correctness labels.

The extracted subgraph includes four types of nodes: the target student, its neighboring exercise nodes, the target exercise, and its neighboring student nodes. A one-hot vector is assigned to a node as its initial node feature, with 0 and 1 assigned to the target student and exercise, respectively, while 2 and 3 are assigned to the neighboring students and exercises, respectively. These one-hot vectors are used as initial node features in the graph-based correctness prediction step.

Rather than using an embedding layer to transform exercise IDs into fixed-dimensional vectors as in some transformer-based KT models, IGKT utilizes KCs such as skills and tags to connect exercises that share these concepts, as illustrated in Fig. 3, allowing the model to capture the relationships between exercises efficiently and improve its ability to predict student performance correctly.

In the EdNet dataset, each exercise belongs to a part and contains one or more tags. Additional exercise information is defined as a KC, and an additional edge between exercises is created based on the co-occurrence of the KC. Table 1 lists the criteria for creating new edge types. Edges can be classified into student-exercise and exercise-exercise interactions.

3.5 Graph-based correctness prediction

In the final step, the correctness of the target interaction is predicted by passing the constructed subgraph through GNNs. To implement the proposed IGKT framework, three alternative models have been created and compared to determine the effectiveness of integrating the timestamp and graph information. The first model, IGKT-Basic, utilizes R-GCN layers and excludes timestamp information. The second model, IGKT-TS, uses R-GCN layers and includes normalized timestamps as message passing weights, with higher weights assigned to the interactions that occurred more recently. The third model, IGKT-GAT, incorporates EGAT layers and uses correctness and normalized timestamp vectors as edge features, calculating attention coefficients from the node and edge vectors.

Figure 4 presents the structure of the GNN layers in IGKT-GAT and IGKT-TS. Both models utilize correctness and timestamp information in their GNN layers; however, IGKT-GAT incorporates this information using an attention mechanism within a single layer. A thicker edge line in the figure indicates a larger message passing weight; nodes connected by edges with larger weights pass more information.

Figure 5 illustrates the message passing at each node level in greater detail. For the Edge Graph Attention (EGAT) layer, an attachment coefficient is computed from each node pair (x_i, x_j) and its edge features e_{ij}; the node embeddings are aggregated using the attention coefficient α_{ij} as a weight. For the R-GCN, the node embeddings are aggregated using the normalized timestamp ts_{ij} as a weight.

Each GNN layer is applied to derive the target node representations. Equation (2) shows how the R-GCN [29] layers

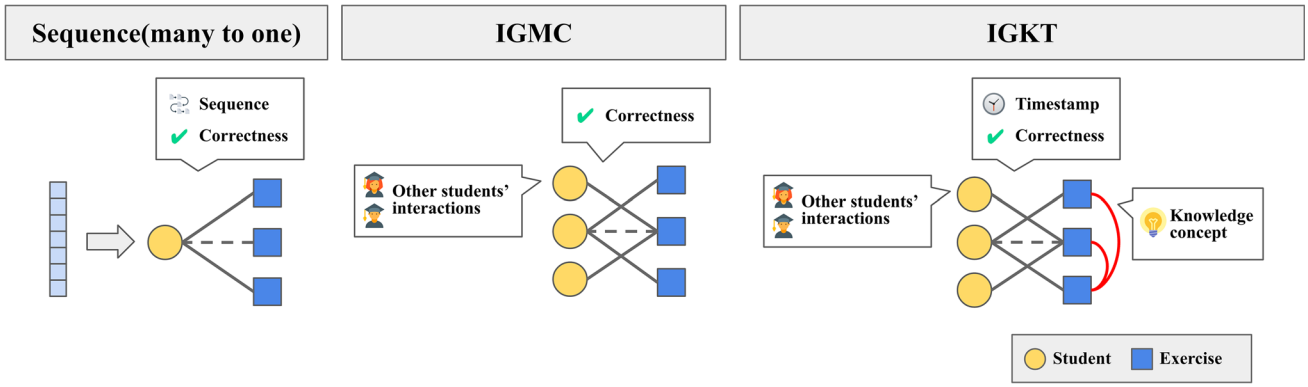

Fig. 3 Comparison of learning methods among sequence-based, IGMC (without timestamp and KCs), and IGKT (with timestamp and KCs) models

Table 1 Edge types and relations for the datasets

Edge type	Relation	EdNet	ASSIST2017
0	student-exercise	correctness=0	correctness=0
1	student-exercise	correctness=1	correctness=1
2	exercise-exercise	same part	same problem type
3	exercise-exercise	tag co-occurrence=1	same skill
4	exercise-exercise	tag co-occurrence=2	-
5	exercise-exercise	tag co-occurrence≥3	-

An edge type of 1 indicates that the student solved the relevant exercise correctly, whereas an edge type of 0 indicates that it was incorrectly solved. Types 2 and above represent specific relationships between exercises based on KCs. The specific labels used and their meanings may vary depending on the configuration of the analyzed dataset

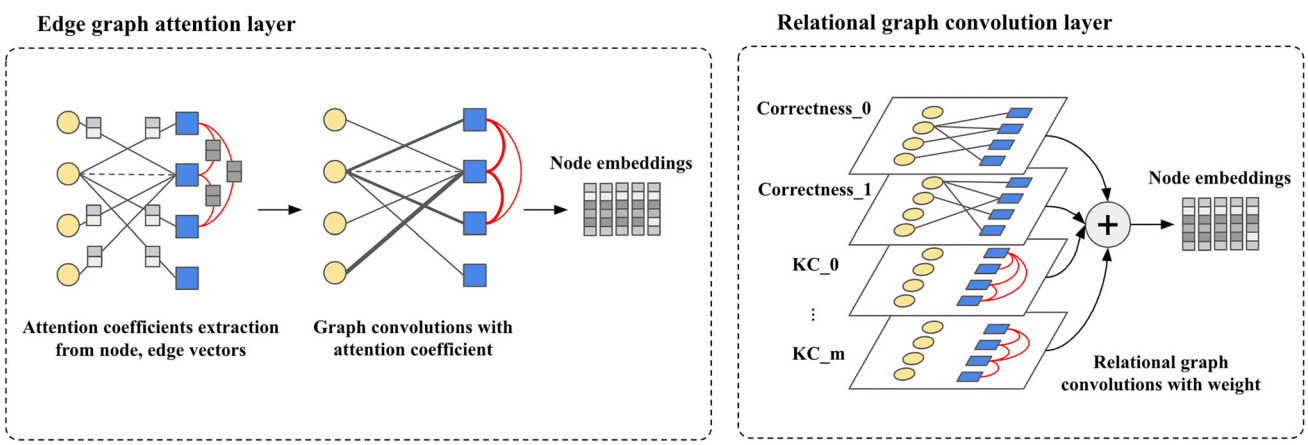

Fig. 4 Comparison of graph convolutional networks in IGKT-GAT and IGKT-TS

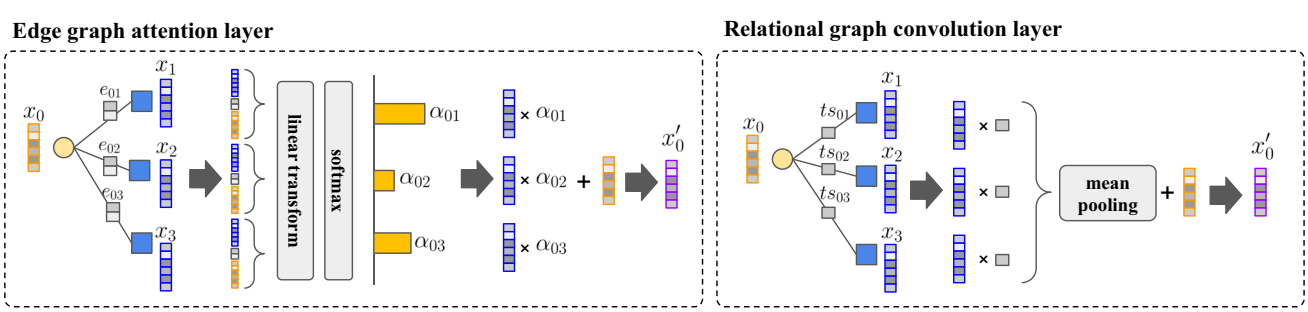

Fig. 5 Comparison of message passing of graph convolutional networks in IGKT-GAT and IGKT-TS

in IGKT-Basic aggregate neighbor node representations and update target node representations as,

$$x_i^{(l)} = \sigma\left(W_0^{(l)} x_i^{(l-1)} + \sum_{r \in R} \sum_{j \in \mathcal{N}_i^r} \frac{1}{\|\mathcal{N}_i^r\|} W_r^{(l)} x_j^{(l-1)}\right), \quad (2)$$

where $x_i^{(l)}$ denotes the representations of node i from the $l-th$ layer and $x_i^{(0)}$ is initialized with the one-hot vector created in the previous subgraph extraction step. $\mathcal{N}_i^r$ denotes the set of neighbors of node i by relation r. $W_r^{(l)}$ denotes the learnable graph convolutional weight used to receive messages from neighboring nodes connected by relationship r in the $l-th$ layer. $W_0^{(l)}$ denotes the learnable weight for the self-loop, and σ denotes the activation function. In our implementation, the Exponential Linear Unit (ELU) is used.

The IGKT-TS model is designed to address the limitation of the IGKT-Basic not considering chronological information. IGKT-TS integrates normalized timestamps as weights in the R-GCN layer. Normalized timestamps are calculated as shown in (3):

$$\begin{aligned} ts_{abs} &= \mid ts_{target} - ts \mid \\ ts_{norm} &= 1 - \frac{ts_{abs} - min(TS_{abs})}{max(TS_{abs}) - min(TS_{abs})}, \end{aligned} \quad (3)$$

where ts_{target} denotes the timestamp of the target interaction and ts denotes the timestamp we want to normalize. ts_{abs} is the distance between the target interaction timestamp and the timestamp to be normalized. TS_{abc} denotes the set of distances between the target interaction timestamp and the timestamps in a subgraph. Finally, subtracting the min-max normalized ts_{abs} values of all interactions inside the subgraph from 1 gives us the normalized timestamps. ts_{norm} represents the normalized timestamp, which assigns a higher weight to interactions that occurred more recently and a lower weight to interactions that occurred earlier. Message passing in IGKT-TS is weighted by these normalized timestamps, as shown in (4):

$$x_i^{(l)} = \sigma\left(W_0^{(l)} x_i^{(l-1)} + \sum_{r \in R} \sum_{j \in \mathcal{N}_i^r} \frac{ts_{norm\,ij}}{\|\mathcal{N}_i^r\|} W_r^{(l)} x_j^{(l-1)}\right). \quad (4)$$

This equation is similar to (2), the difference is adapting $ts_{norm\,ij}$, which denotes the normalized timestamp of the interaction between nodes i and j.

In addition, we propose a variant model called IGKT-GAT that utilizes a graph attention layer. In IGKT-GAT, correctness and normalized timestamps are concatenated into 2-dimensional vectors and are provided as edge features as shown in (5):

$$e_{ij} = concat(c_{ij}, ts_{norm\,ij}). \quad (5)$$

Here, e_{ij}, c_{ij}, and $ts_{norm\,ij}$ denote the edge feature vector, correctness, and normalized timestamp between node i and j, respectively. Unlike R-GCN, all edges participate in message passing with different weights based on the attention coefficient. Equation 6 describes how the attention coefficient is computed in EGAT [31] layer as,

$$\alpha_{ij} = \frac{exp(\sigma(W_\alpha[W_h x_i || W_h x_j || W_e e_{ij}])}{\sum_{k \in \mathcal{N}_i} exp(\sigma(W_\alpha[W_h x_i || W_h x_k || W_e e_{ik}])}. \quad (6)$$

Here, α_{ij} is the attention coefficient between node i and j. The W_α, W_h, and W_e denote the attention weight, weight for node features, and weight for edge features, respectively, which are all learnable parameters. The edge feature vector e_{ij} is used to compute the graph attention coefficient, which enables message passing to consider both correctness and timestamps. Equation 7 shows the adoption of the graph attention coefficient to update the node embedding as,

$$x_i^{(l)} = \sigma\left(W_0^{(l)} x_i^{(l-1)} + \sum_{j \in \mathcal{N}_i} \alpha_{ij} W_1^{(l)} x_j^{(l-1)}\right), \quad (7)$$

where $W_0^{(l)}$ and $W_1^{(l)}$ denote the learnable weights in the $l-th$ layer for the self-loop and neighboring nodes, respectively.

After deriving node representations from each GNN layer, we concatenate the representations of node i from each layer and assign them to its final representation, denoted by x_i'. Based on the final representation set, we extract the final target student representation x_s' and exercise representation x_e'. Then, the target student and exercise representations are concatenated, and the final scalar value $\hat{c}$, which represents the predicted probability of correctness, is obtained through the fully connected layer. A sigmoid function is used to normalize $\hat{c}$ between 0 and 1. The equation for this is written as,

$$\begin{aligned} x_i' &= concat(x_i^1, x_i^2, ..., x_i^L) \\ g &= concat(x_s', x_e') \\ \hat{c} &= Sigmoid(Wg). \end{aligned} \quad (8)$$

3.6 Model training

Given that the task of our model is binary classification, binary cross entropy (BCE) loss was used as in (9):

$$\mathcal{L}_{BCE} = -\frac{1}{n} \sum_{i=0}^{n} (c_i \log(\hat{c}_i) + (1 - c_i) \log(1 - \hat{c}_i)), \quad (9)$$

where n denotes the number of samples, c the ground-truth correctness and $\hat{c}$ the predicted probability of correctness. In addition, the Adam [32] optimizer is used.

4 Experiment

This section describes the experimental environment and settings. The proposed IGKT models were compared with other models to answer the following research questions:

RQ1 Does the inductive graph-based KT (IGKT) approach achieve improved performance over existing models?
RQ2 What is the most effective method for incorporating time information in IGKT?
RQ3 How does the combination of KCs and subgraph components impact the performance of IGKT?
RQ4 How does the performance of the model change as the length of the interaction sequence varies?

4.1 Datasets

Our experiment was conducted using two datasets selected from different domains: EdNet [33] and ASSISTments2017 Competition (ASSIST2017) [34].

EdNet comprises English problem solving record data collected from Korean users in Santa[2], a multi-platform AI English tutoring service, and is one of the largest public datasets in education. It has a structure with up to six skills in one exercise problem and seven problem types in total. Each problem type is categorized into one of three listening parts, a grammar part, a vocabulary part, and two reading comprehension parts.

ASSIST2017 contains data on solving math problems collected from middle and high school students through the ASSISTments tutoring system[3] and was released through the 2017 ASSISTments Data mining Competition[4]. One skill is considered per practice example, belonging to one of 16 problem types such as multiple choice, fill in the blank, and short answer. Both datasets concern students, exercises, skills, problem types, and timestamps. For EdNet, 30M data points (30% of the entire set), were sampled and used for easy experimentation. Table 2 presents the summary statistics of the two datasets.

Table 2 Dataset statistics

Dataset	EdNet	ASSIST2017
skill type	Multiple	Single
#student	117,345	1,709
#exercise	13,517	3,162
#skill	188	102
#skill per exercise	2.292	1.00
#exercise type	7	16
#interaction per student	255.66	551.68
#interaction	30,000,000	942,816

4.2 Baselines

- **DKT** [18] is the model proposed by the first study that applied deep learning to KT. It uses an RNN to track the knowledge state of students. Students' knowledge states are learned with a single weight matrix without the distinction of concepts.
- **DKVMN** [19] is influenced by a memory network [35] and tracks the student's status of knowledge in the form of an external memory matrix. Unlike DKT, it models students' knowledge states for individual KCs.
- **SAKT** [5] is the model proposed by the first study that applied the self-attention mechanism to KT. It has a single encoder-based structure and uses the combined interaction of exercises and responses as keys and values, respectively.
- **SSAKT** is a stack variant of SAKT and applies self-attention before inputting an exercise into a query to solve the problem. Its performance degrades when attention blocks are stacked multiple times [6].
- **SAINT+** [20] is a model that integrates the elapsed and lag time information into the SAINT model, which was designed to improve its performance by separating exercises and responses from SAKT and learning with encoders and decoders, respectively.
- **SAINT+(skill)** is a variant of the SAINT+ model that includes skill (tag) information. This model was included in the experiments for comparison with our models, which also use skill information.
- **GKT** [8] is the model proposed by the first study that applied a GNN to KT. It constructs a graph by modeling the relationships between KCs, and updates the knowledge state corresponding to the related concept using the student's learning record. However, it is limited in modeling as it associates only one concept per exercise.
- **IGMC** [16] is the model proposed for handling large-scale graphs in recommender systems. It has a structure that is robust against changes in graph size by extracting a partial graph and demonstrates a high prediction performance without node features and using only structural information. In our implementation, only two edge types were used, distinguishing between correct and incorrect responses.

[2] https://www.aitutorsanta.com/kr/
[3] https://new.assistments.org/
[4] https://sites.google.com/view/assistmentsdatamining

- **PEBG** [36] is a model that uses pre-trained exercise embeddings as side information, such as the exercise difficulty and relationships between questions and skills. This model uses a product-based neural network to process the side information and incorporates pre-trained embeddings into existing RNN-based models

4.3 Experiment settings

Evaluation Metric To compare the prediction performance of IGKT with those of other baseline methods, the *accuracy* (ACC) and *area under the receiver operating characteristic curve* (AUC) were employed as evaluation metrics.

Training Setting All datasets were divided into training (80%) and testing (20%) sets in chronological order. For the EdNet dataset, in which the exercise concerns up to six KCs, the upper two concepts were vectorized and integer encoding was performed to implement DKT, DKVMN, and GKT. The number of preprocessed KCs was 379. Our model was implemented using PyTorch [37] and DGL [38], and the BCE loss was used with the Adam optimizer to consider both the ACC and AUC. Our implementation is available in an online repository[5].

For fair comparisons between the models, we set the hyperparameters to be as identical as possible. For example, a learning rate of {0.002, 0.001, 0.0005}, batch size of {64, 128}, and the number of hidden dimensions of {32} were applied consistently.

4.4 Results

Table 3 presents a summary of the IGKT models and baseline performances. Our key observations are as follows.

First, as an answer to **RQ1**, IGKT-GAT exhibited the highest performance for both of the datasets, without exception. Among the baseline models, SAINT+(skill) performed the best on the EdNet dataset, and IGMC (Primitive inductive graph-based model excluding time information and KCs) performed the best on the ASSIST2017 dataset. IGKT-Basic exhibited a higher performance than SAINT+ but a lower performance than SAINT+(skill). Taken together, the results indicate that considering timestamps in the task is important, as in both IGKT-TS and IGKT-GAT, which incorporate timestamp information into the graph. Furthermore, it is interesting to note that, despite the differences in the datasets, all IGKT models performed better than the baselines on the ASSIST2017, without exception, and two IGKT models (IGKT-TS, IGKT-GAT) on the Ednet datasets, without exception. On EdNet, SAINT+(skill), which was designed specifically for EdNet by the same author, showed a better performance than IGKT-Basic for accuracy. This difference in outcome may have occurred due to the data from ASSIST2017 being denser, containing over twice the number of interactions per student, as shown in Table 2.

Table 3 Accuracy and ROC-AUC performance of baselines and IGKT

Dataset	EdNet		ASSIST2017	
Metric	ACC	AUC	ACC	AUC
DKT	0.6868	0.6771	0.6789	0.7005
DKVMN	0.6859	0.6732	0.6795	0.6952
SAKT	0.6355	0.6407	0.6575	0.6742
SSAKT	0.6466	0.6361	0.6536	0.6765
SAINT+	0.7083	0.7471	0.6807	0.7263
SAINT+(skill)	<u>0.7381</u>	<u>0.7724</u>	0.6718	0.6951
GKT	OOT	OOT	0.6858	0.7125
PEBG	0.6169	0.6438	<u>0.7061</u>	<u>0.7644</u>
IGMC	0.6209	0.6184	0.6898	0.7572
IGKT-Basic	0.7236	0.7805	0.7188	0.7919
IGKT-TS	0.7433	0.7883	0.7212	0.7701
IGKT-GAT	**0.7605**	**0.8142**	**0.8028**	**0.8852**
Improvement	2.95%	5.14%	14.07%	14.45%

The best results are shown in bold, and the best baseline results are underlined. Each experiment was repeated five times and the *mean* is reported. "OOT" indicates that the experiment ran out of time

Second, the training time of GKT, which was the first to apply a GNN to KT, was too long to learn the EdNet dataset (having run out of time). GKT composes a dense KC graph that corresponds to the number of students and has a structure designed to learn node embeddings through an interaction sequence. It has a time complexity of $O(\#knowledge_concepts^2 * \#students * sequence_length)$ for the entire training. The original author conducted experiments using data that limited the number of students to 1,000. Compared with the author's original experiments [8], EdNet required more than 1,000 times the amount of computation. In this context, it exhibited an unrealistic running time in our experimental environment, requiring more than 6 h per epoch.

Third, on the ASSIST2017 dataset, PEBG outperformed the other baseline models. However, PEBG performed poorly on the EdNet dataset, suggesting that the proposed pre-training approach in PEBG has a different level of effectiveness depending on the characteristics of the datasets. Conversely, our proposed method is effective on both datasets, highlighting the consistent effectiveness of our model.

4.5 Ablation study

To answer **RQ2** and **RQ3**, we performed an ablation study to verify the importance of incorporating other students' interactions and two types of KCs (skill, problem type) into the

[5] https://github.com/venzino-han/igkt

KT model. To determine the contribution of each component in the proposed model, we tested several slimmed-down models that removed other students' interactions and KCs. More specifically, in each dataset, there are two types of KCs: skill (tag) and problem type (part). Thus, in addition to using them all, we created three additional variants: two variants that uses only one KC type (-type), (-skill), and a variant that excludes all KCs (-w/o KC). In the original versions (IGKT-Basic, IGKT-TS, and IGKT-GAT), both types of KC and all other components were used. We also included a variant (-single) that excludes other students' interactions to see if they contribute to performance improvement. Table 4 shows these variants of IGKT.

Table 5 presents the ablation study results. Among the IGKT variants, IGKT-GAT-type achieved the best performance on the EdNet dataset, and IGKT-GAT achieved the best performance on the ASSIST2017 dataset. These results suggest that, for the GNN layers, the EGAT-based approach, which utilizes time information and correctness as edge features, is effective for our tasks.

We can observe a decrease in performance when excluding other students' interactions and when excluding all KCs, confirming that both factors contribute to performance improvement. Another interesting point is that IGKT-Basic and IGKT-TS, both of which employ R-GCN, performed the worst when excluding all KCs. In contrast, IGKT-GAT, which employs EGAT, performed the worst when other students' interactions were excluded. Taken togher, the results indicate that the two GNNs (i.e., R-GCN, EGAT) learn by focusing on different characteristics.

The results on the ASSIST2017 dataset indicate that incorporating both types of KCs leads to significant improvements in terms of both evaluation metrics. The significant performance gap between all IGKT variants and IGMC suggests that the proposed method with edge augmentation effectively captures the characteristics of the problem in the graph-based approach. However, on the EdNet dataset, contrary to our expectation, using only one type of KC outperformed using both types. We speculate that this phenomenon may have been caused by differences in the relationship characteristics between the two KCs; the analysis section presents a more detailed analysis.

5 Analysis

5.1 Sequence length effect

To answer **RQ4**, we analyzed the performance changes of each model with different maximum sequence lengths on the EdNet dataset. As shown in Table 6, SAKT and SAINT+ using the attention mechanism, demonstrated improved performance as the sequence length increased. However, SAKT's performance decreased when the length increased to 256, likely because it does not incorporate additional time-related features and is unable to filter outdated data.

IGKT-Basic and IGKT-TS achieved the best performance at a length of 32, whereas IGMC and IGKT-GAT showed the best performance at a length of 128. We interpret these results from three perspectives. First, the decreasing perfor-

Table 4 Variants of IGKT by components

Variants	GNN	KC1(problem type)	KC2(skill)	Timestamp	Other students' interactions
IGKT-Basic-single	R-GCN	✓	✓	✗	✗
IGKT-Basic-w/o KC	R-GCN	✗	✗	✗	✓
IGKT-Basic-type	R-GCN	✓	✗	✗	✓
IGKT-Basic-skill	R-GCN	✗	✓	✗	✓
IGKT-Basic	R-GCN	✓	✓	✗	✓
IGKT-TS-single	R-GCN + edge weight	✓	✓	✓	✗
IGKT-TS-w/o KC	R-GCN + edge weight	✗	✗	✓	✓
IGKT-TS-type	R-GCN + edge weight	✓	✗	✓	✓
IGKT-TS-skill	R-GCN + edge weight	✗	✓	✓	✓
IGKT-TS	R-GCN + edge weight	✓	✓	✓	✓
IGKT-GAT-single	EGAT	✓	✓	✓	✗
IGKT-GAT-w/o KC	EGAT	✗	✗	✓	✓
IGKT-GAT-type	EGAT	✓	✗	✓	✓
IGKT-GAT-skill	EGAT	✗	✓	✓	✓
IGKT-GAT	EGAT	✓	✓	✓	✓

Table 5 Ablation study results on KC utilization in IGKT variants

Dataset	EdNet		ASSIST2017	
Metric	ACC	AUC	ACC	AUC
IGKT-Basic-single	0.7162	0.7186	0.7062	0.7695
IGKT-Basic-w/o KC	0.6647	0.6154	0.6841	0.7556
IGKT-Basic-type	0.7157	0.7812	0.7168	0.7861
IGKT-Basic-skill	**0.7302**	**0.7836**	**0.7298**	0.7865
IGKT-Basic	0.7236	0.7805	0.7188	**0.7919**
IGKT-TS-single	0.7143	0.7117	0.7120	0.7739
IGKT-TS-w/o KC	0.6579	0.6081	0.6847	0.7453
IGKT-TS-type	**0.7476**	0.7887	0.7142	**0.7824**
IGKT-TS-skill	0.7444	**0.7907**	0.7153	0.7772
IGKT-TS	0.7433	0.7883	**0.7212**	0.7701
IGKT-GAT-single	0.7221	0.7419	0.7103	0.7518
IGKT-GAT-w/o KC	0.7585	0.8096	0.7486	0.8402
IGKT-GAT-type	**0.7617**	**0.8179**	0.7989	0.8814
IGKT-GAT-skill	0.7616	0.8149	0.7945	0.8742
IGKT-GAT	0.7605	0.8142	**0.8028**	**0.8852**

The best results for each IGKT variant are shown in bold

mance of IGKT-Basic as the sequence length increases can be attributed to IGKT-Basic assigning the same weight to learning previous interactions regardless of their temporal order. Additionally, the case of IGKT-TS shows that using normalized timestamps as edge weights is insufficient for handling longer sequences. Second, the optimal performance of IGMC at longer sequence lengths may be caused by the need for longer sequence information, as it does not consider the relationship between exercises but only the overall correctness of the interaction. Third, IGKT-GAT achieved the best performance at a sequence length of 128, demonstrating the effectiveness of our EGAT-based method in utilizing longer sequences. However, its performance decreased at a sequence length of 256, indicating that further feature engineering, such as the use of timestamps as in SAINT+, may be necessary to effectively utilize longer sequences.

Figures 6 and 7 illustrate the distribution of student sequence lengths for each dataset. In both datasets, we found that a small number of users had very long sequences. For the EdNet dataset, we found that the distribution was more skewed. If we separate each student by the length of their learning sequences, only 5.6% of students in ASSIST2017 had sequence lengths below 64, while 63.1% of students had learning sequences below 64. These distributive differences of the two datasets may have contributed to the differing performance improvements.

5.2 Comparing performance by problem type

Figure 8 shows the percentage of interactions in each dataset that belongs to each problem type. We found that 3 out of 7 problem types accounted for 70.4% of all interactions in EdNet, and 4 out of 16 problem types accounted for 80.7% in ASSIST2017. Considering the characteristics of these datasets, we expected differences in performance based on problem type. Hence, we evaluated the difference in performance based on problem type in our test results. Figures 9 and 10 show the AUC performance for each problem type in the test datasets. In the case of ASSIST2017, problem type 6 did not exist in the test dataset. For most problem types, IGKT-GAT performed well with very little training data, except for problem types 3 and 11. In EdNet, IGKT-GAT performed better on all other problem types than on problem type 5, which was the most dominant with 41.1%. These results suggest that our model is robust to problem types with insufficient training data and can be effective in situations with imbalanced data. We attribute this property of our model to the fact that it does not directly utilize the semantics contained in the problem types, but rather reflects them in the form of a graph structure.

5.3 Comparing models' number of learnable parameters

The number of model parameters is an important factor in determining training time and memory usage. We compared the number of parameters in the baselines and the three variants of IGKT models to analyze not only the performance

Table 6 AUC performance results obtained while varying the sequence length on the EdNet dataset

Length of sequence	16	32	64	128	256
SAKT	0.6407	0.6404	0.6407	**0.6742**	0.6410
SAINT+	0.7247	0.7334	0.7471	0.7533	**0.7543**
SAINT+(skill)	0.7582	0.7686	0.7724	0.7772	**0.7783**
IGMC	0.6105	0.6136	0.6184	**0.6231**	0.6171
IGKT-Basic	**0.7828**	**0.7828**	0.7805	0.7808	0.7620
IGKT-TS	0.7893	**0.7942**	0.7883	0.7858	0.7722
IGKT-GAT	0.8022	0.8126	0.8142	**0.8178**	0.8028

The best results for each baseline and IGKT variant are shown in bold

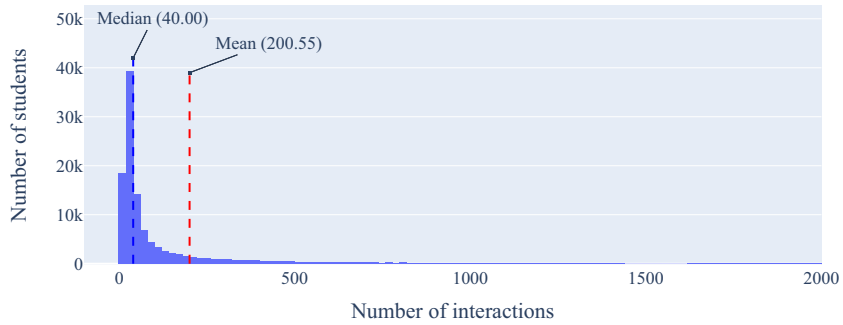

Fig. 6 EdNet student interaction length distribution. The maximum length in the histogram has been limited to 2000 for better visualization

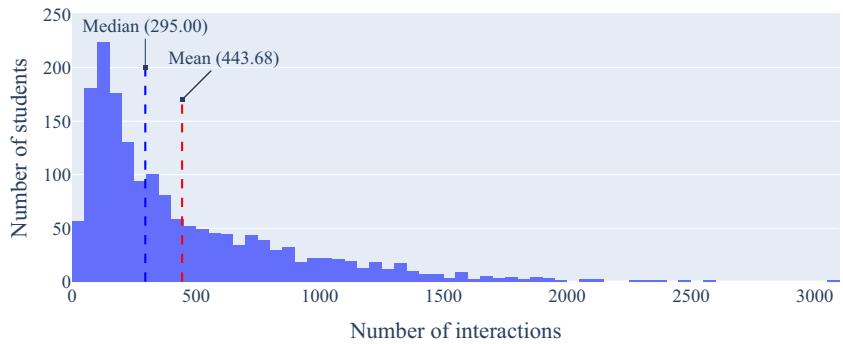

Fig. 7 ASSIST2017 student interaction length distribution

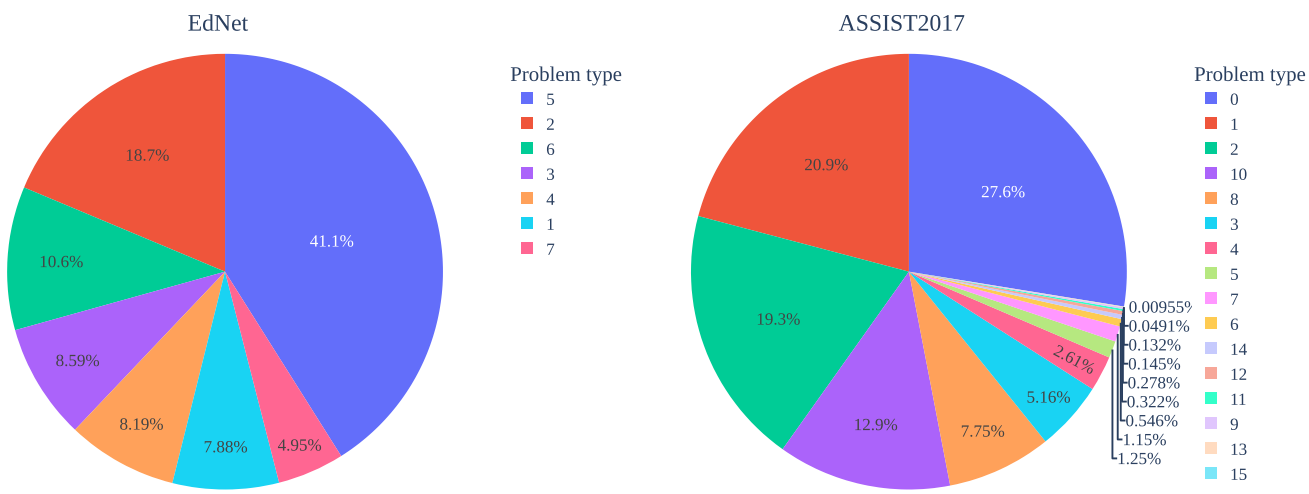

Fig. 8 Percentage of interactions by problem type in the EdNet and ASSIST2017 datasets

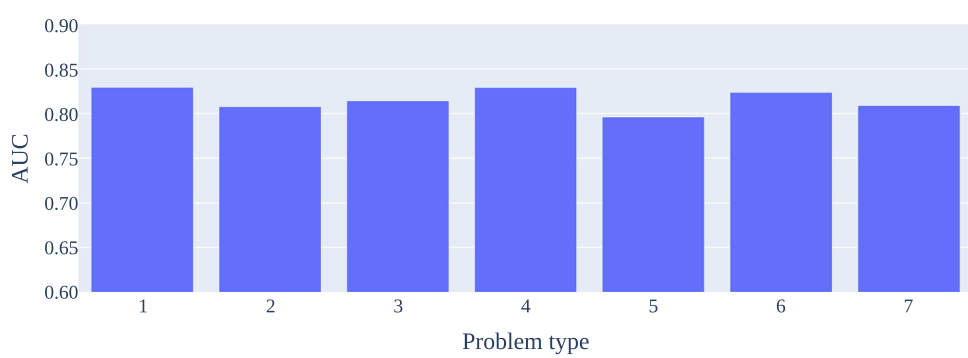

Fig. 9 Test AUC performance on the EdNet dataset

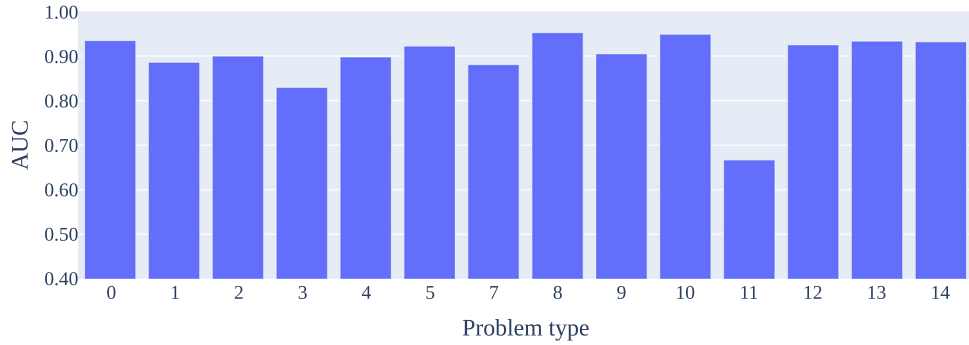

Fig. 10 Test AUC performance on the ASSIST2017 dataset

of the models but also the cost of the models. Figure 11 shows the number of parameters for the models we used in our experiments. SAINT+, a transformer-based self-attention model, required the most number of parameters because it uses multiple encoders and decoders, and multi-head attentions. However, our proposed graph-based models, IGKT variants, have an average of 48k parameters, which is more than the existing KT models, but reasonable considering their high performance.

5.4 Associations between knowledge concepts

To further investigate the performance differentiation between the two datasets used in the experiment, we analyzed the two datasets in terms of KC overlapping. Figure 12 shows how the skill (tag) overlaps several problem types (parts) within each dataset. The x-axis indicates the number of problem types over which the skill is observed, and the y-axis indicates the number of skills.

Our analysis of the EdNet and ASSIST2017 datasets revealed that the skills in EdNet are often isolated to a single problem type (114 out of 188), whereas skills in ASSIST2017 frequently co-occur across multiple problem types, suggesting that the skills in EdNet represent sub-information regarding a specific problem type whereas skills in ASSIST2017 represent a different type of information that is not bound to a particular problem type. Our experiments show that using both types of KCs in the EdNet dataset results in performance degradation, indicating that IGKT is effective at integrating KCs that are independent of each other, indicating that the relationships between different types of KCs can impact model performance, and that considering the relationships is important when designing KT models.

We also compared the mutual information (MI) between different KCs in the two datasets. MI is a measure of the dependence between two variables; a higher MI indicates a stronger relationship between them. Utilizing MI in graph-based methods is also known to be effective [39]. Equation 10 expresses how the MI between X and Y is calculated; the higher the probability that x and y appear together, the higher the MI:

$$MI(X;Y) = \sum_{y \in Y} \sum_{x \in X} P(x,y) \, log \left(\frac{P(x,y)}{P(x)P(y)} \right). \tag{10}$$

We calculated the MI between the different KC types in the two datasets. The MI between the KCs was 1.3031 in EdNet and 0.6538 in ASSIST2017. These results suggest that using KCs with a high MI may negatively impact the performance of the IGKT compared with using a single KC.

From another perspective, in the EdNet dataset, the correlations between problem types were relatively low, indicating that most students have different probabilities of correctness for each problem type. The heatmap in Fig. 13 illustrates the correlations of correctness between problem types in each dataset; bluer squares indicate lower correlations, suggesting that the KCs represented by each problem type are isolated from each other.

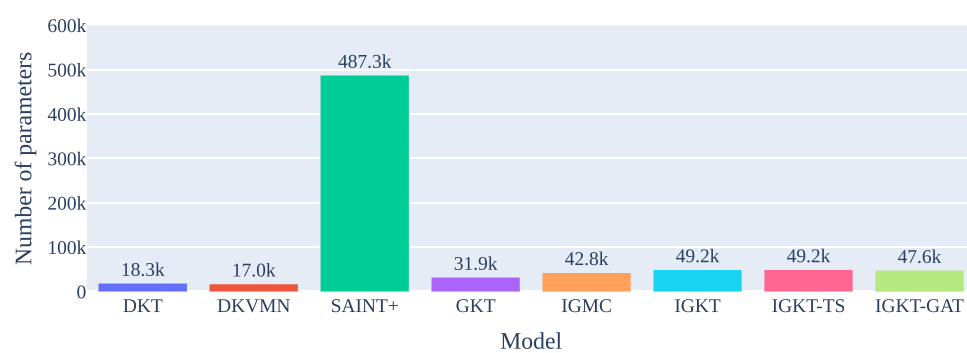

Fig. 11 Comparing the number of parameters among the models

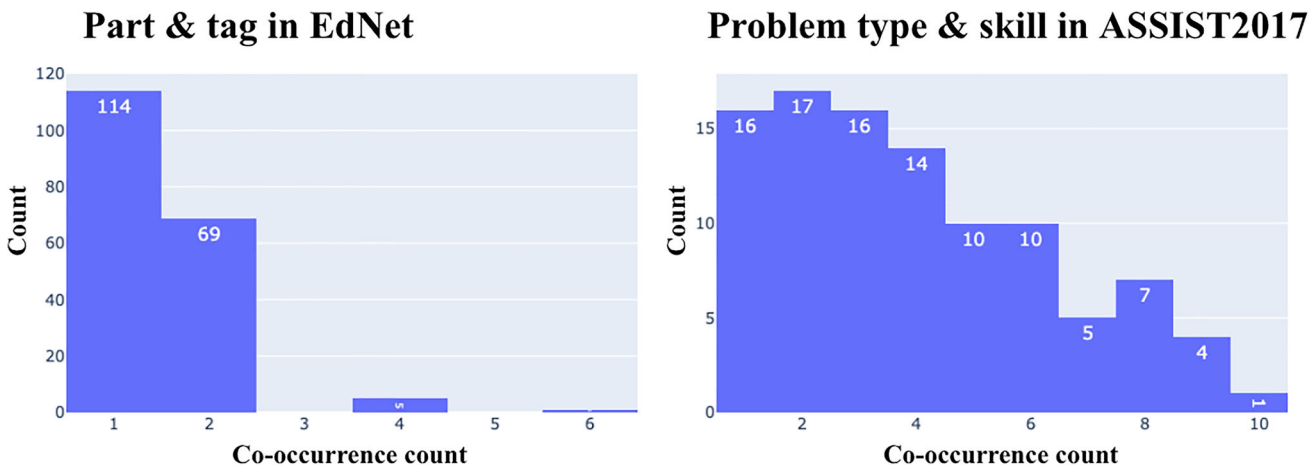

Fig. 12 Co-occurrences between different types of KCs

Consequently, the performance differences between IGMC and IGKT were more pronounced in the EdNet dataset in our main experiment. IGKT achieved a higher performance by adding edges that represent KC relevance, enabling it to capture the differences in knowledge levels for each isolated KC. Conversely, in the ASSIST2017 dataset, which had higher inter-problem type correlations, the correctness of one problem type can be better used to predict the correctness of other problem types.

6 Discussion

Our study is the first attempt to apply an inductive graph-based method to KT. Our work has demonstrated that it is effective to integrate other students' interactions and timestamps into a graph. Our proposed framework can utilize a variety of data collected in online learning platforms without limitations, as additional information existing between students and exercises can be utilized as edge features, and various relationships between exercises can be utilized as additional edges. To take advantage of these characteristics, designing online courses requires logging various information during the learning process and modeling the relationship between exercises.

Our framework can be applied to online education applications to predict student knowledge states and to recommend appropriate learning activities. Recommending appropriate exercises is a powerful way toward maximizing learning with limited interactions. Our IGKT (Inductive Graph-based Knowledge Tracing) method based on the proposed frame-

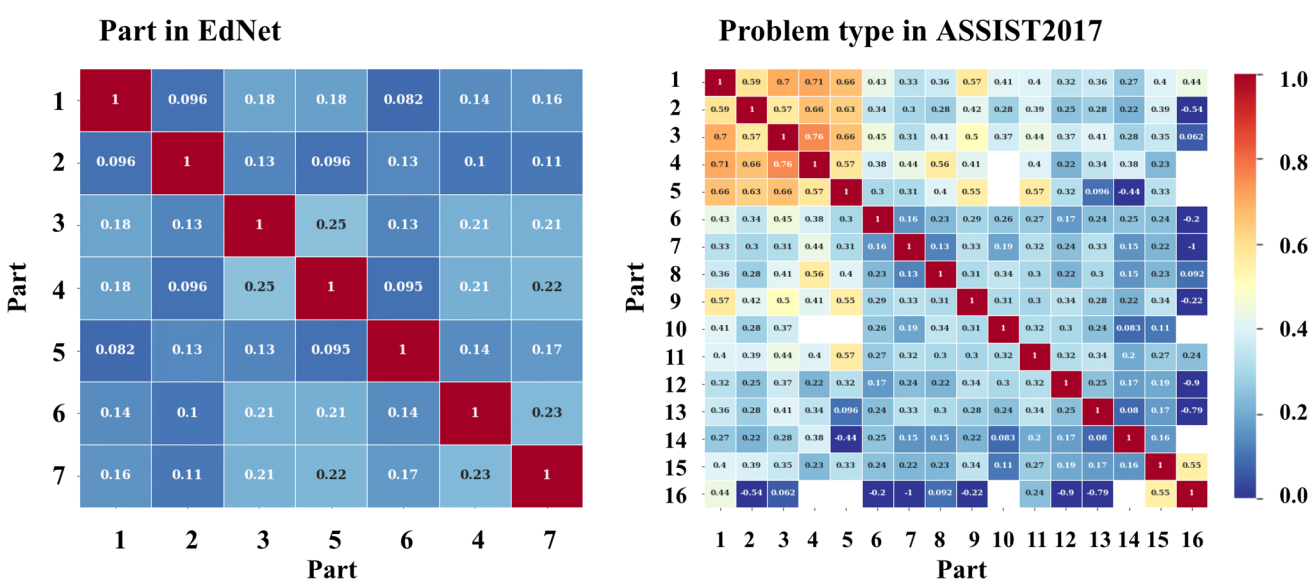

Fig. 13 Heatmaps show the correlations of the percentage of correctly answered questions with other KCs in the datasets

work not only shows the largest performance improvements, but also has advantageous characteristics in real-world applications. First, the method is also effective in the presence of KC imbalance and different interaction sequence lengths for students, which is useful when dealing with unrefined datasets. Second, our method does not require an excessive number of parameters, so it can be easily applied to web applications targeting a large number of users. Finally, the method does not use sensitive personal information such as age, gender, and education besides the user's learning history thereby making it immune to personal data misuse.

Despite the aforementioned advantages and practicality, our work has limitations that can be explored in future works. The first limitation of our method is that it does not utilize relationships among KCs such as prerequisites or similarities. The datasets we used for our experiments did not include explicit relationships between KCs, thus we were unable to model additional relationships between KCs. If relationships between KCs are available, they can be reflected in the graph structure by modeling KCs as additional nodes and relationships between KCs as edges. The second limitation is that we were not able to leverage the relationships between students. If we had access to additional information about the students, we would be able to add relationships between students to the graph. Recently, there has been some research on social-based recommendations [10, 11, 40, 41], which could be leveraged to model these relationships. The final limitation of our method is its explainability and the limitation of utilizing textual data. Adopting the graph-based explainable models [42–44] can help clarify factors that influence the student's state of knowledge. A method of adding a layer that identifies important nodes among those included in the subgraph increases the explainability of our models. Also, the data we used did not contain the original text of the exercise, thereby limiting the ability of our study to utilize text. However, by utilizing exercise text, additional semantics can be extracted and used to improve the performance and explainability of the models [45, 46]. Furthermore, interest in using hypergraphs has been growing in various domains [47–49]. By representing various KCs and user groups as hyperedges, the proposed method can be further developed to address some of these limitations.

7 Conclusion

In this study, we introduced the method of IGKT, which leverages student-exercise interactions and KC information in a graph-based model. We extended conventional methods, which only exploit the interaction sequence of a single student, to include the interactions of multiple students. We added edges representing KCs to learn associations between exercises and used the R-GCN and EGAT layers to obtain representation vectors for the target student and exercise from the subgraph. In addition, timestamp data was incorporated into the GNN layers. We then conducted predictions by measuring the probability of the correct answers.

Our extensive experiments demonstrated that the proposed IGKT method, which incorporates timestamp data, outperforms state-of-the-art KT models such as SAINT+. We also demonstrated that the proposed method is more efficient in terms of time complexity. Furthermore, we observed that incorporating KCs as edges in the model significantly improves performance and justified the environmental values, ensuring the best performance with multiple KC types. The study findings clearly show that the proposed framework and the derived models represent a promising step towards maximizing student achievements in online educational applications.

Acknowledgements This work was supported by a grant of the KAIST-KT joint research project through AI2XL Laboratory, Institute of convergence Technology, funded by KT [G01220547, Graph neural network-based educational content personalization recommendation system for self-directed learning]

References

1. Abdelrahman G, Wang Q, Nunes B (2023) Knowledge Tracing: A Survey. ACM Comput Surv 55(11)
2. Song X, Li J, Cai T, Yang S, Yang T, Liu C (2022) A survey on deep learning based knowledge tracing. Knowl Based Syst 258:110036
3. Wu Z, Li M, Tang Y, Liang Q (2020) Exercise recommendation based on knowledge concept prediction. Knowl Based Syst 210:106481
4. Liu Q, Tong S, Liu C, Zhao H, Chen E, Ma H, Wang S (2019) Exploiting Cognitive Structure for Adaptive Learning. Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '19, page 627-635, New York, NY, USA, Association for Computing Machinery
5. Pandey S, Karypis G (2019) A self-attentive model for knowledge tracing. In 12th International Conference on Educational Data Mining, EDM 2019, pages 384–389. International Educational Data Mining Society
6. Choi Y, Lee Y, Cho J, Baek J, Kim B, Cha Y, Shin D, Bae C, Heo J (2020) Towards an appropriate query, key, and value computation for knowledge tracing. In Proceedings of the Seventh ACM Conference on Learning @ Scale, L@S '20, page 341-344, New York, NY, USA. Association for Computing Machinery
7. Wang X, Zheng Z, Zhu J, Yu W (2023) What is wrong with deep knowledge tracing? attention-based knowledge tracing. Appl. Intell 53(3):2850–2861
8. Nakagawa H, Iwasawa Y, Matsuo Y (2019) Graph-based knowledge tracing: Modeling student proficiency using graph neural network. In 2019 IEEE/WIC/ACM International Conference on Web Intelligence (WI), pages 156–163

9. Gao C, Zheng Y, Li N, Li Y, Qin Y, Piao J, Quan Y, Chang J, Jin D, He X, Li Y (2023) A survey of graph neural networks for recommender systems: Challenges, methods, and directions. ACM Trans Recomm Syst 1(1)
10. Gao C, Wang X, He X, Li Y (2022) Graph neural networks for recommender system. In Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining, WSDM '22, page 1623-1625, New York, NY, USA, Association for Computing Machinery
11. Huang C, Xu H, Xu Y, Dai P, Xia L, Lu M, Bo L, Xing H, Lai X, Ye Y (2021) Knowledge-aware coupled graph neural network for social recommendation. Proceedings of the AAAI Conference on Artificial Intelligence 35(5):4115–4122
12. Wang X, He X, Wang M, Feng F, Chua TS (2019) Neural graph collaborative filtering. Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR'19, page 165-174, New York, NY, USA, Association for Computing Machinery
13. Wu L, Li J, Sun P, Hong R, Ge Y, Wang M (2022) Diffnet++: A neural influence and interest diffusion network for social recommendation. IEEE Trans Knowl Data Eng 34(10):4753–4766
14. He Z, Li W, Yan Y (2022) Modeling knowledge proficiency using multi-hierarchical capsule graph neural network. Appl Intell 52(7):7230–7247
15. Song X, Li J, Lei Q, Zhao W, Chen Y, Mian A (2022) Bi-CLKT: Bi-Graph Contrastive Learning based Knowledge Tracing. Knowl Based Syst 241:108274
16. Zhang M, Chen Y (2020) Inductive matrix completion based on graph neural networks
17. Corbett AT, Anderson JR (1994) Knowledge tracing: Modeling the acquisition of procedural knowledge. User modeling and user-adapted interaction 4(4):253–278
18. Piech C, Bassen J, Huang J, Ganguli S, Sahami M, Guibas LJ, Sohl-Dickstein J (2015) Deep knowledge tracing. Advances in neural information processing systems, 28
19. Zhang J, Shi X, King I, Yeung DY (2017) Dynamic key-value memory networks for knowledge tracing. In Proceedings of the 26th international conference on World Wide Web, pages 765–774
20. Shin D, Shim Y, Yu H, Lee S, Kim B, Choi Y (2021) Saint+: Integrating temporal features for ednet correctness prediction. In LAK21: 11th International Learning Analytics and Knowledge Conference, LAK21, page 490-496, New York, NY, USA. Association for Computing Machinery
21. Wang C, Ma W, Zhang M, Lv C, Wan F, Lin H, Tang T, Liu Y, Ma S (2021) Temporal Cross-Effects in Knowledge Tracing. In Proceedings of the 14th ACM International Conference on Web Search and Data Mining, WSDM '21, page 517-525, New York, NY, USA. Association for Computing Machinery
22. Abdelrahman G, Wang Q (2023) Deep Graph Memory Networks for Forgetting-Robust Knowledge Tracing. IEEE Trans Knowl Data Eng 35(8):7844–7855
23. Tong H, Wang Z, Zhou Y, Tong S, Han W, Liu Q (2022) Introducing problem schema with hierarchical exercise graph for knowledge tracing. In Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '22, page 405-415, New York, NY, USA. Association for Computing Machinery
24. Devlin J, Chang MW, Lee K, Toutanova K (2019) BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics
25. Ding X, Han T, Fang Y, Larson E (2022) An approach for combining multimodal fusion and neural architecture search applied to knowledge tracing. Appl Intell
26. van den Berg R, Kipf TN, Welling M (2018) Graph convolutional matrix completion. In 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining
27. Hamilton WL, Ying R, Leskovec J (2017) Inductive representation learning on large graphs. In Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17, page 1025-1035, Red Hook, NY, USA. Curran Associates Inc
28. Ying R, He R, Chen K, Eksombatchai P, Hamilton WL, Leskovec J (2018) Graph convolutional neural networks for web-scale recommender systems. In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '18, page 974-983, New York, NY, USA. Association for Computing Machinery
29. Schlichtkrull M, Kipf TN, Bloem P, van den Berg R, Titov I, Welling M (2018) Modeling relational data with graph convolutional networks. In Aldo Gangemi, Roberto Navigli, Maria-Esther Vidal, Pascal Hitzler, Raphaël Troncy, Laura Hollink, Anna Tordai, and Mehwish Alam, editors, The Semantic Web, pages 593–607, Cham, Springer International Publishing
30. Veličković P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y (2018) Graph attention networks. In International Conference on Learning Representations
31. Wang Z, Chen J, Chen H (2021) Egat: Edge-featured graph attention network. In International Conference on Artificial Neural Networks, pages 253–264. Springer
32. Kingma DP, Ba J (2015) Adam: A method for stochastic optimization. In Yoshua Bengio and Yann LeCun, editors, International Conference on Learning Representations
33. Choi Y, Lee Y, Shin D, Cho J, Park S, Lee S, Baek J, Bae C, Kim B, Heo J (2020) Ednet: A large-scale hierarchical dataset in education. In Ig Ibert Bittencourt, Mutlu Cukurova, Kasia Muldner, Rose Luckin, and Eva Millán, editors, Artificial Intelligence in Education, pages 69–73, Cham. Springer International Publishing
34. Cully A, Demiris Y (2020) Online knowledge level tracking with data-driven student models and collaborative filtering. IEEE Transactions on Knowledge and Data Engineering 32(10):2000–2013
35. Weston J, Chopra S, Bordes A (2015) Memory networks. In 3rd International Conference on Learning Representations, ICLR 2015
36. Liu Y, Yang Y, Chen X, Shen J, Zhang H, Yu Y (2020) Improving knowledge tracing via pre-training question embeddings. In Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI'20
37. Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Killeen T, Lin Z, Gimelshein N, Antiga L, Desmaison A, Kopf A, Yang E, DeVito Z, Raison M, Tejani A, Chilamkurthy S, Steiner B, Fang L, Bai J, Chintala S (2019) Pytorch: An imperative style, high-performance deep learning library. In: Wallach H, Larochelle H, Beygelzimer A, d'Alché-Buc F, Fox E, Garnett R (eds) Advances in Neural Information Processing Systems 32. Curran Associates Inc, pp 8024–8035
38. Wang M, Zheng D, Ye Z, Gan Q, Li M, Song X, Zhou J, Ma C, Yu L, Gai Y, Xiao T, He T, Karypis G, Li J, Zhang Z (2019) Deep graph library: A graph-centric, highly-performant package for graph neural networks
39. Peng Z, Huang W, Luo M, Zheng Q, Rong Y, Xu T, Huang J (2020) Graph Representation Learning via Graphical Mutual Information Maximization. In Proceedings of The Web Conference 2020, WWW '20, page 259-270, New York, NY, USA. Association for Computing Machinery
40. Zhu Z, Gao C, Chen X, Li N, Jin D, Li Y (2022) Inhomogeneous Social Recommendation with Hypergraph Convolutional

Networks. 2022 IEEE 38th International Conference on Data Engineering (ICDE)
41. Liao J, Zhou W, Luo F, Wen J, Gao M, Li X, Zeng J (2022) Sociallgn: Light graph convolution network for social recommendation. Inf Sci 589:595–607
42. Ying Z, Bourgeois D, You J, Zitnik M, Leskovec J (2019) Gnnexplainer: Generating explanations for graph neural networks. Adv Neural Inf Process Syst 32
43. Mei X, Zhou Y, Zhu C, Wu M, Li M, Pan S (2023) A disentangled linguistic graph model for explainable aspect-based sentiment analysis. Knowl Based Syst 260:110150
44. Li Y, Liu L, Wang G, Du Y, Chen P (2022) Egnn: Constructing explainable graph neural networks via knowledge distillation. Knowl Based Syst 241:108345
45. Zhu X, Li Z, Wang X, Jiang X, Sun P, Wang X, Xiao Y, Yuan NJ (2022) Multi-modal knowledge graph construction and application: A survey. IEEE Transactions on Knowledge and Data Engineering, pages 1–20
46. Chen X, Zhang N, Li L, Deng S, Tan C, Xu C, Huang F, Si L, Chen H (2022) Hybrid transformer with multi-level fusion for multimodal knowledge graph completion. In Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '22, page 904-915, New York, NY, USA, Association for Computing Machinery
47. Antelmi A, Cordasco G, Polato M, Scarano V, Spagnuolo C, Yang D (2023) A survey on hypergraph representation learning. ACM Comput Surv, Just Accepted
48. Çatalyürek Ü, Devine K, Faraj M, Gottesbüren L, Heuer T, Meyerhenke H, Sanders P, Schlag S, Schulz C, Seemaier D, Wagner D (2023) More recent advances in (hyper)graph partitioning. ACM Comput Surv 55(12)
49. Wang Z, Chen CH, Zheng P, Li X, Song W (2022) A hypergraph-based approach for context-aware smart product-service system configuration. Comput Ind Eng 163:107816

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Donghee Han received his B.S. degree in industrial design in 2016 and M.S. degree in data science in 2023, from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea. He worked as a designer in LG Hausys from 2016 to 2018, and as a researcher at Tmax A&C from 2018 to 2020. He was a data engineer at Buzzvil from 2020 to 2022. He is currently pursuing his Ph.D. degree in Data Science from KAIST. His research interests include recommender systems, graph neural networks and natural language processing.

Daehee Kim received his B.S. degree in systems management engineering and computer engineering from Sungkyunkwan University, Suwon, South Korea, in 2022. He is currently pursuing an M.S. degree in Data Science from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea. His research interests include graph neural network, knowledge tracing and recommender systems.

Minsu Kim received a B.S. degree in math and computer science from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2020. He received an M.S. degree in electrical engineering from KAIST, Daejeon, South Korea, in 2022. He is currently pursuing a Ph.D. degree in industrial and system engineering at KAIST. His research interests include deep learning and reinforcement learning.

Keejun Han received his B.S. degree in information technology from Australian National University, in 2010, and M.S. and Ph.D degrees in knowledge service engineering from KAIST, in 2013 and 2019, respectively. He worked as a staff engineer for Samsung Electronics from 2019 and 2021. He was a researcher with Intelligence Convergence Division, ETRI from 2021 to 2022. He is now an assistant professor at School of Computer Engineering, Hansung University, located in Seoul, South Korea. His research interests include natural language processing, knowledge graph embedding and smart factory.

Mun Yong Yi received his Ph.D. degree in information systems from the University of Maryland, College Park, MD, USA, in 1998. Before joining KAIST in 2009, he taught at the University of South Carolina and the University of Maryland. He is currently a professor in the Department of Industrial and Systems Engineering, Korea Advanced Institute of Science and Technology (KAIST). His research has been published in a number of academic journals, including Information Systems Research, Decision Sciences, Decision Support Systems, Information and Management, Journal of Information Science, International Journal of Human-Computer Studies, Journal of Applied Psychology, Scientific Reports, IEEE TRANSACTIONS ON CONSUMER ELECTRONICS, and IEEE ACCESS. His research interests include both behavioral and technical aspects of information technology, including business analytics, human-computer interaction, and knowledge engineering. He currently serves as an associate editor for the International Journal of Human-Computer Studies and a senior editor for AIS Transactions on Human-Computer Interaction.

Authors and Affiliations

Donghee Han[1] · Daehee Kim[1] · Minsu Kim[2] · Keejun Han[3] · Mun Yong Yi[2]

Donghee Han
handonghee@kaist.ac.kr

Daehee Kim
dh.kim@kaist.ac.kr

Minsu Kim
min-su@kaist.ac.kr

[1] Graduate School of Data Science, Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea

[2] Department of Industrial and Systems Engineering, Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea

[3] Division of Computer Engineering, Hansung University Seoul, Republic of Korea